API Security Assessment Report

Prepared by: Charmaine Mangorima
Date: November 21, 2025
Target Application: VAmPI API

Executive Summary

This report analyzes the provided API Swagger specification to identify three critical security flaws, following the OWASP API Security Top 10 methodology. Each finding includes an explanation of the security risk and recommended mitigations.

The identified flaws include:

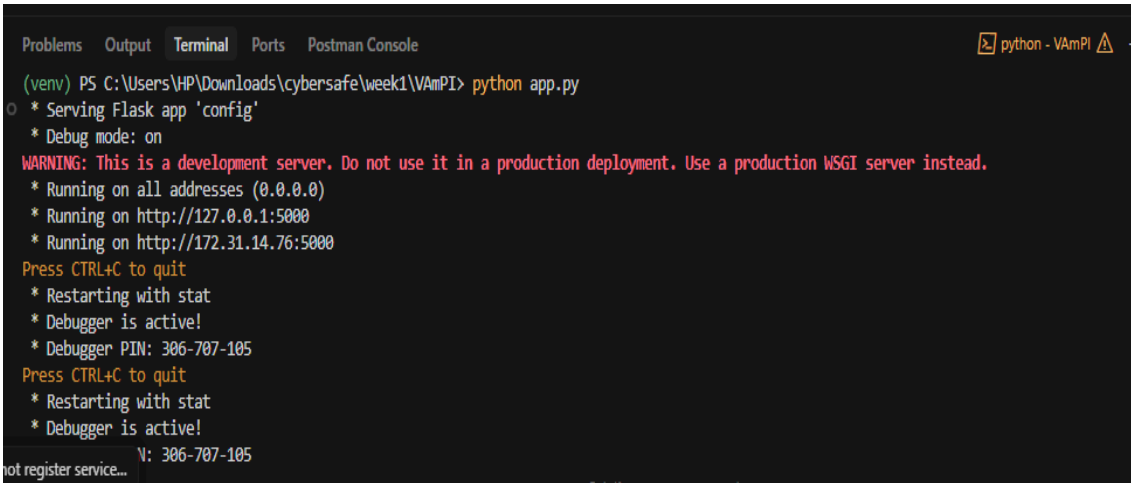
- Broken Object Level Authorization
- Excessive Data Exposure
- Lack of Access Controls on Sensitive Endpoints

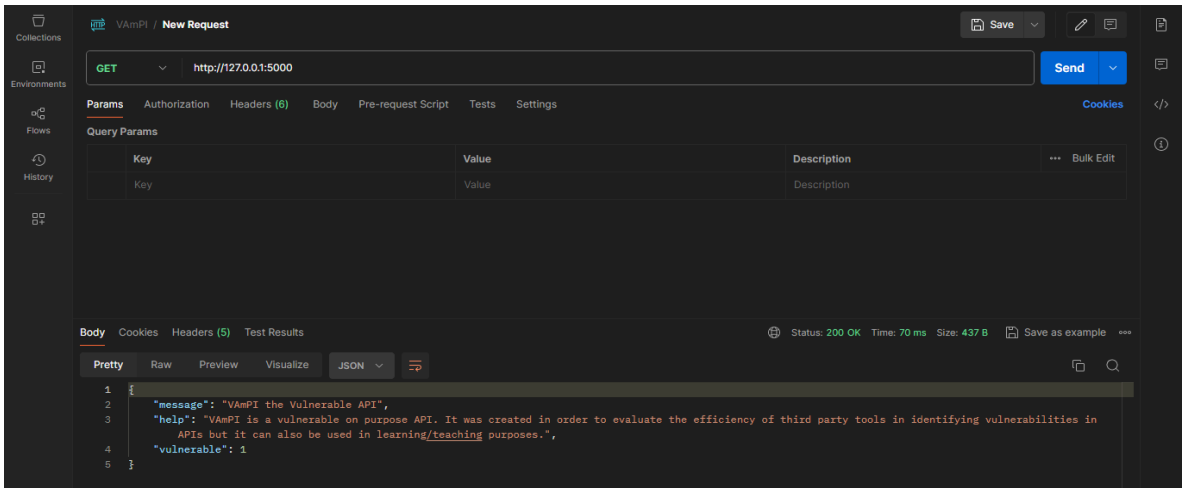
Tools used

- VS Code/Cursor (to start the server)
- Postman (manual API testing)
- Github (https://github.com/erev0s/VAmPI)

Starting the server

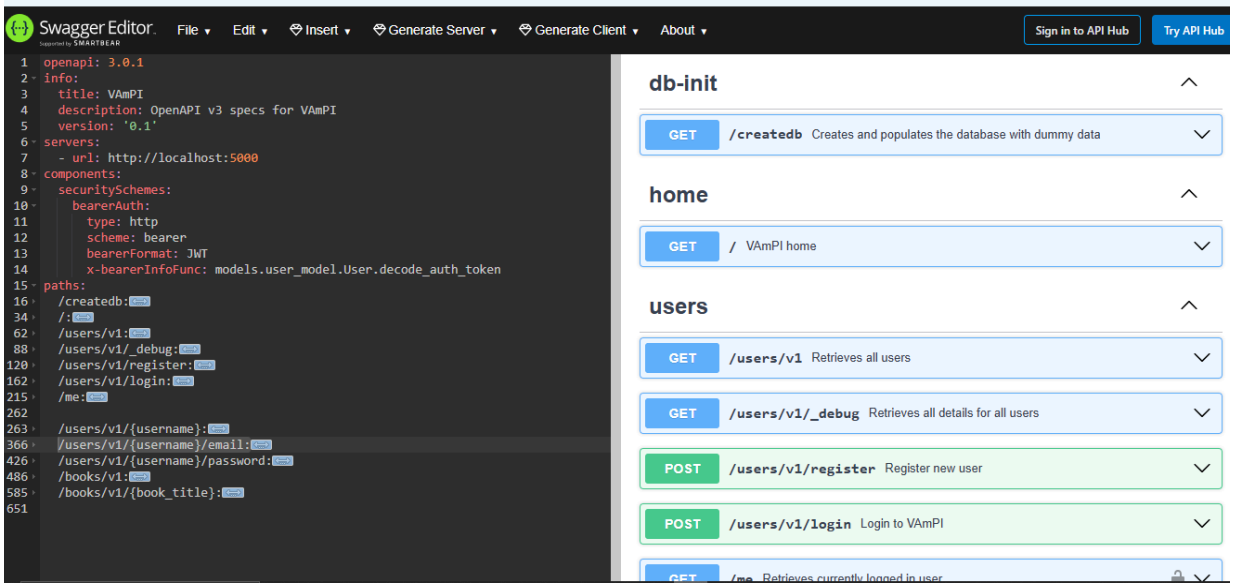
VAmPI is a vulnerable API made with Flask and it includes vulnerabilities from the OWASP top 10 vulnerabilities for APIs. It was created as a vulnerable API to evaluate the efficiency of tools used to detect security issues in APIs. To start this server, I cloned the repository on github, installed all the requirements via **pip install -r requirements.txt** and started the server on my machine and it is running on localhost **http://127.0.0.1:5000** as shown below:





I also used another method by importing the VAmPI swagger specification into Postman.

I made use of Swagger editor to get a clear view of the API endpoints:



Identified Security Flaws

1 Broken Object Level Authorization (BOLA) – OWASP API1

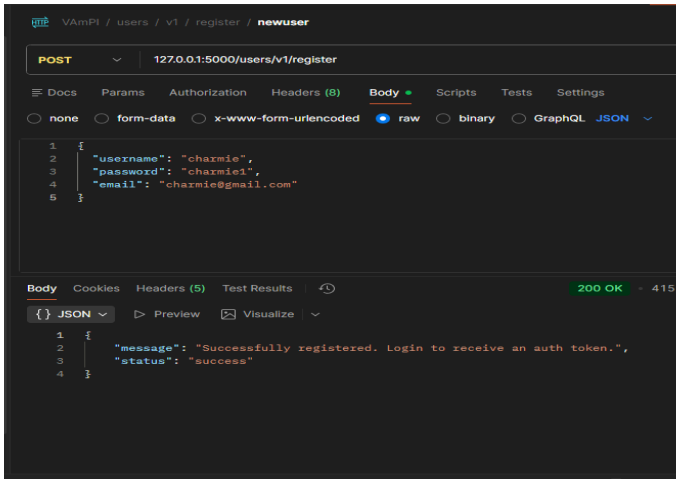
Description:

The endpoint **/books/v1/{book_title}** returns sensitive secret information but relies only on basic bearer authentication. The specification claims that only resource owners may retrieve this

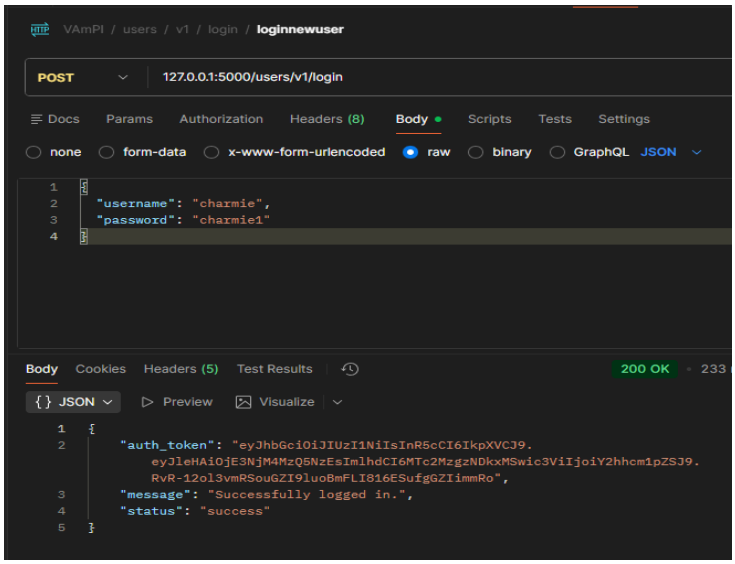
data, but no ownership check or role validation is documented. This allows authenticated users to access other users' secrets.

Evidence:

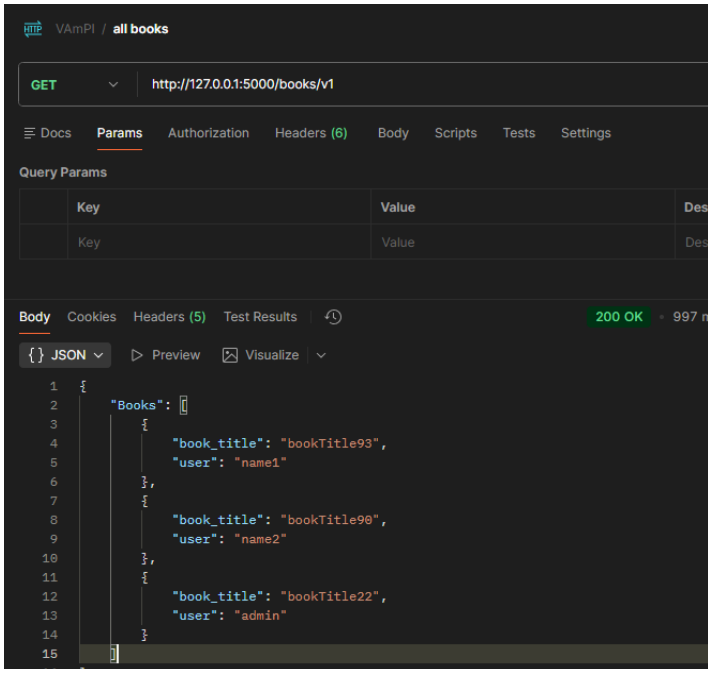
- I created a user (charmie) on the **POST /users/v1/register** endpoint.



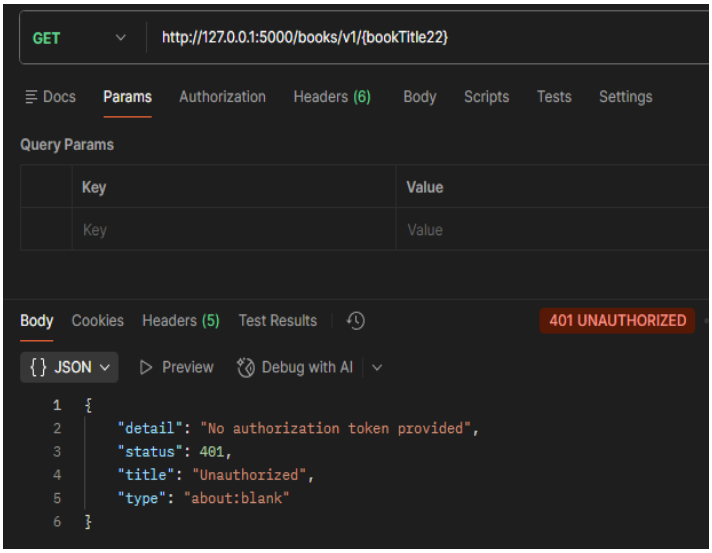
- I successfully logged in as the new user on **POST/users/v1/login** endpoint and got an auth-token in the response.



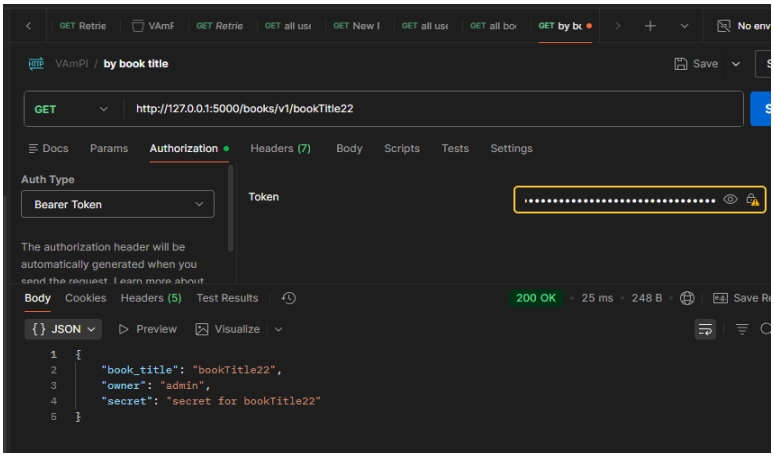
From the end point **/books/v1**, anyone can see the list of all books, even without providing an auth token:



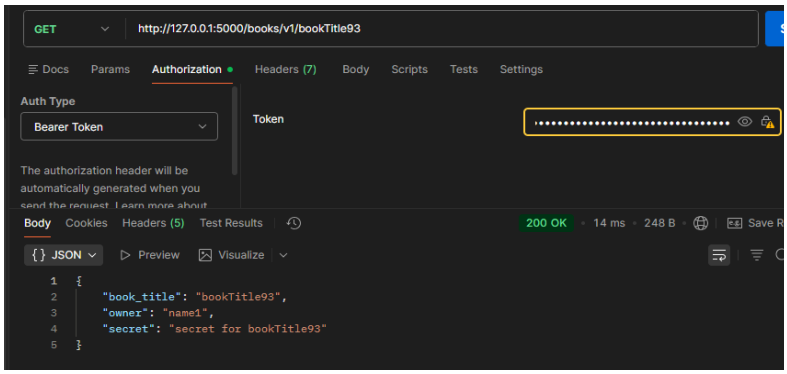
I first tried accessing **/books/v1/bookTitle22** without providing an auth token and it did not bring back the expected secret in the response.



Using the auth token that was sent when the new user logged in as the Bearer Token, I went ahead and accessed another user's (admin) book and their secret via **/books/v1/bookTitle22**, without any authorization checks:

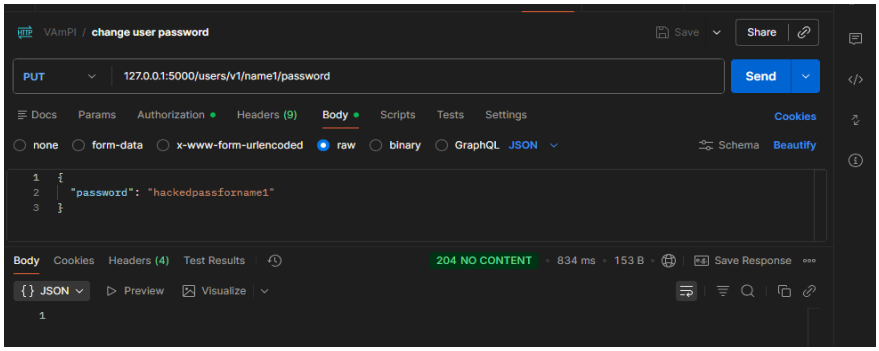


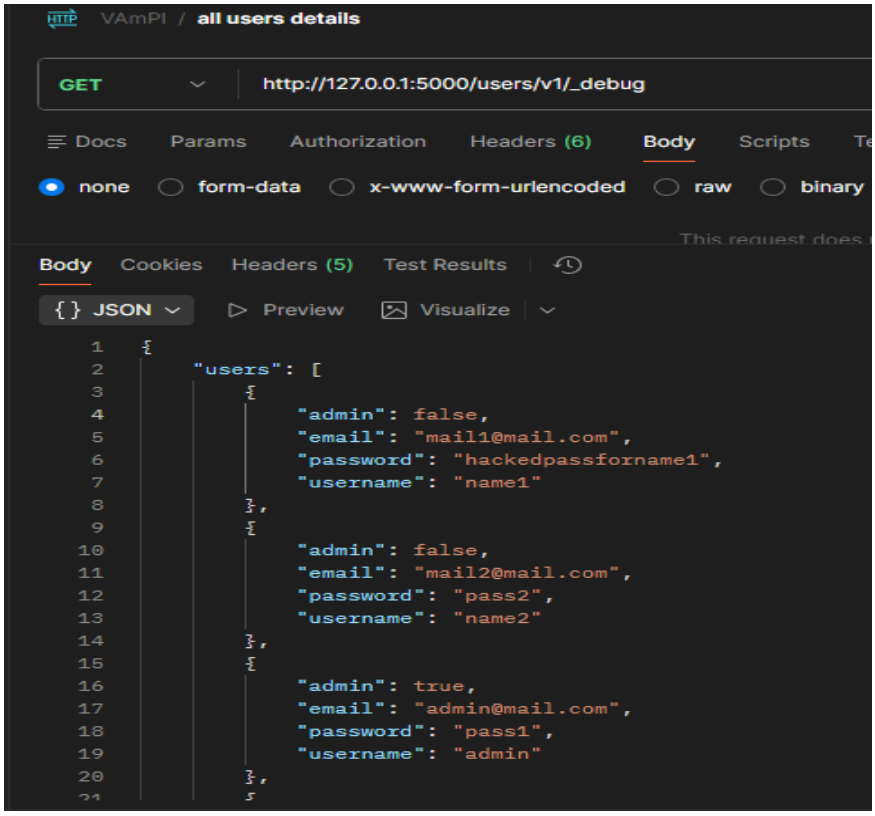
The API shows critical BOLA because the same logged in user is able to access all the other user's books and access their secret. Example **/books/v1/bookTitle93** that belongs to **name1.**



- The endpoint **/users/v1/{username}/password** allows any authenticated user to update the password of any other user simply by changing the **{username}** path parameter. The server does not verify object ownership or enforce role-based authorization. As a result, User A can modify User B's email without permission.

I logged in as the new user that I created and captured the auth token when logging in. I used this token and I was able to change another user's password.





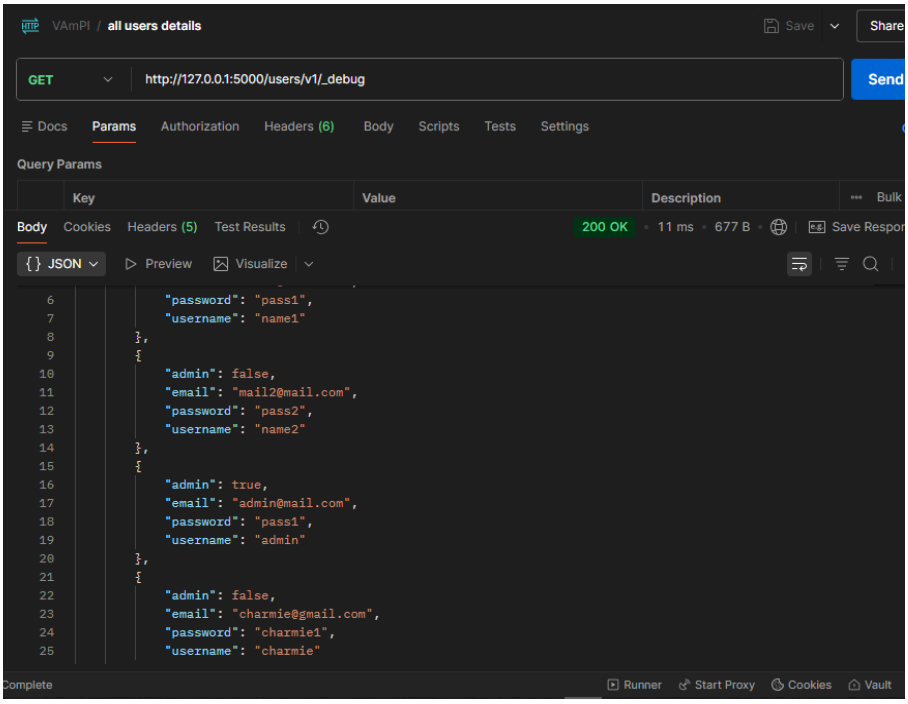
Mitigation:

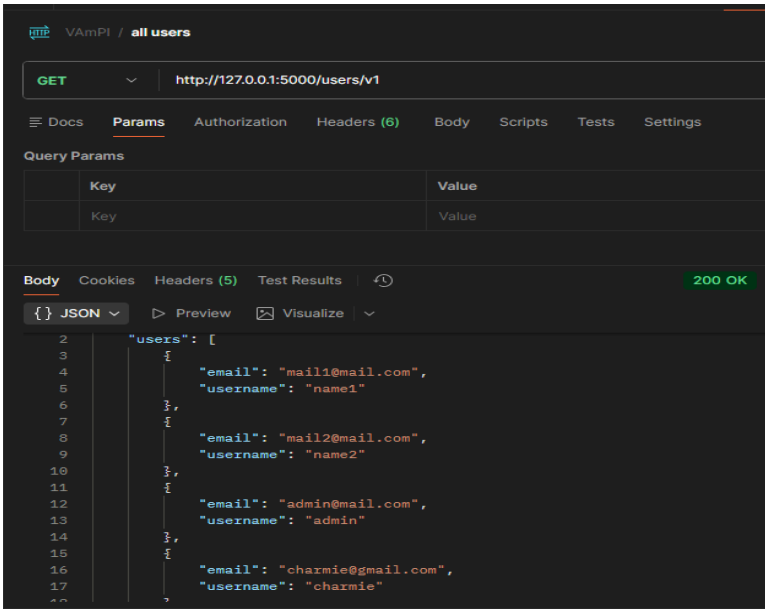
- Implement server-side ownership checks (**check if the resource owner is the current user**).
- Use role-based access control (RBAC) or OAuth2 scopes.
- Update Swagger to specify required authorization scopes.

2 Excessive Data Exposure - Broken Object Property Level Authorization(BOPLA) OWASP API3

Description:
The endpoints **/users/v1/_debug** and **/users/v1** return full user objects including sensitive fields such as password, admin and email and no token was required. This also includes the new user that I just registered. These fields are directly exposed in API responses without filtering. The debug endpoint returns the entire user list. This exposure allows attackers to harvest sensitive data without authentication or with minimal access.

Evidence:





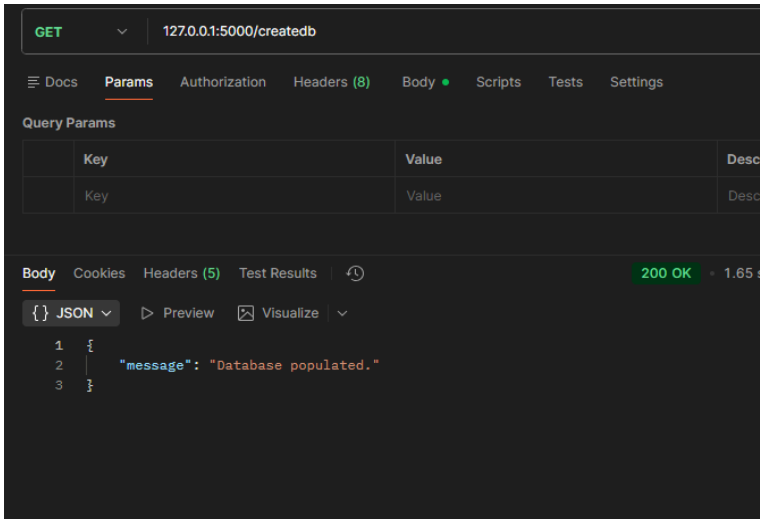
Mitigation:

- Remove sensitive fields from all API responses.
- Disable or restrict debug endpoints from production.
- Implement response filtering to expose only safe fields.

3 Lack of Access Controls on Sensitive Endpoints – Broken Function Level Authorization OWASP API5

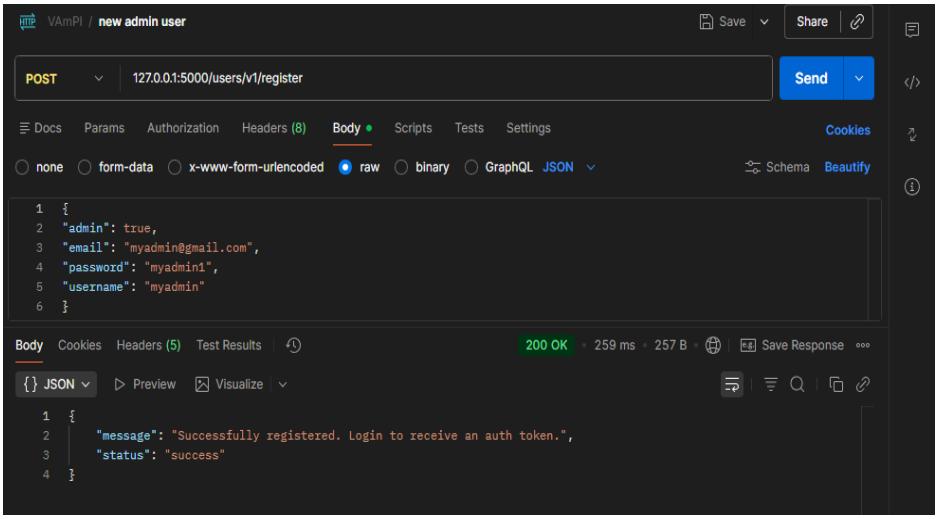
- **Reset Database Without Authentication**

The **/createdb** endpoint allows database creation and populates it without any authentication. As shown in the response message “Database populated.”



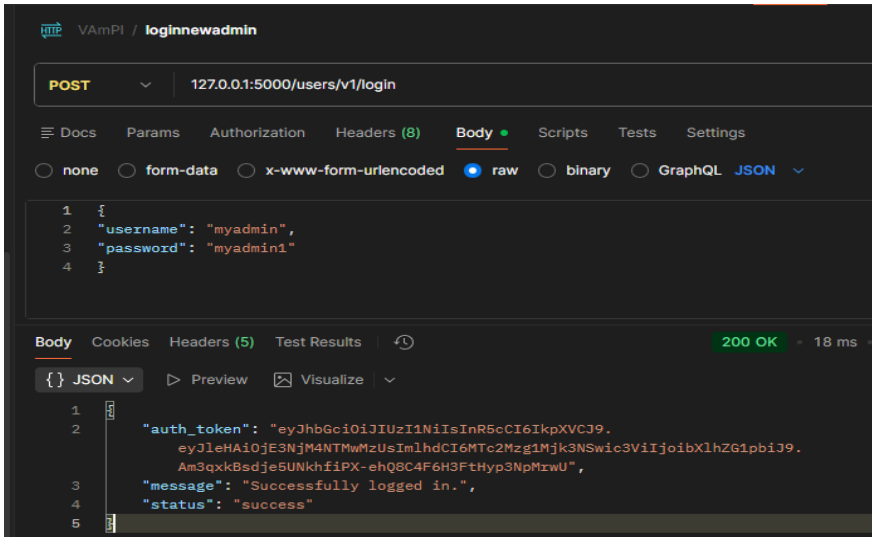
- **Privilege Escalation via User-Controlled Role Assignment**

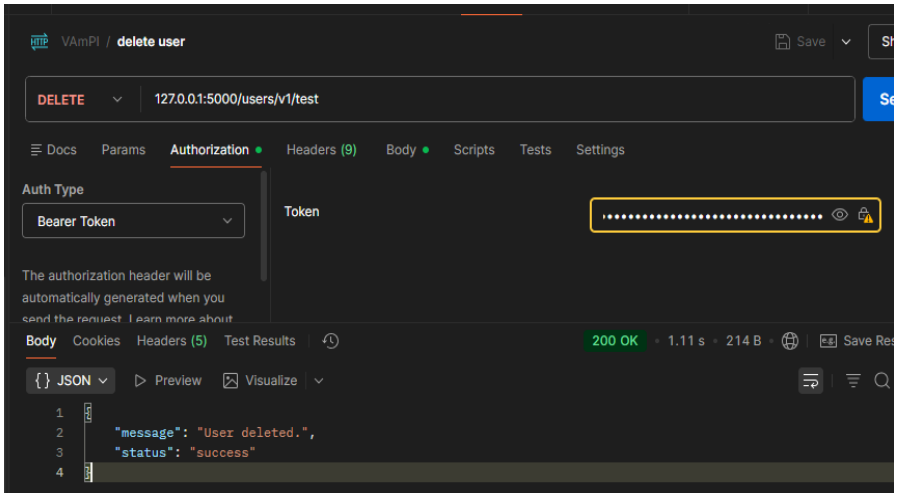
The signup endpoint **/users/v1/register** allows users to set their own authorization level by including **"admin": true** in the request body. There is no server-side validation preventing normal users from registering as administrators. This results in immediate privilege escalation and full compromise of administrative functionality. This means an attacker is able to use admin functions without any authorization checks, for example deleting users.



- **Delete Any User**

Now logged in as the new admin(attacker) and using the admin-only endpoint to delete users(**delete /users/v1/{username}** that relies only on the Bearer Token without enforcing authorization, i managed to delete a test user that i created before. This allows unauthorized destructive operations.





Mitigation:

- Remove development/admin endpoints from production.
- Enforce admin-only role checks on privileged actions like creating admin users and delete.
- Restrict sensitive endpoints via IP whitelisting or environment-based disabling.

Conclusion

The reviewed API specification contains multiple high-risk vulnerabilities. Implementing the recommended mitigations will significantly improve the security posture and protect both user data and backend systems.

What I learnt in the API Security Fundamentals 2025 module

1. Why APIs are high-value targets

APIs are central to modern applications and expose business logic. Because they often handle sensitive data, they are prime targets for attackers.

2. Real world API breaches

The course provided real examples of API breaches, showing how attackers exploit misconfigured or insecure endpoints and the resulting business impact.

3. OWASP API Security Top 10 (2023)

I learnt the major risks including Broken Object-Level Authorization, excessive data exposure, and more along with strategies to prevent them:

API1 Broken Object Level Authorization (BOLA)

This is when APIs often expose endpoints that allow access to objects by id.
If authorization checks are missing or weak, attackers can manipulate object identifiers to access other users' data.

Fix:
Always verify that the authenticated user owns or is allowed to access the requested object and always discuss authorization rules during API design.

API2 Broken Authentication

Weak authentication mechanisms allow attackers to compromise authentication tokens or exploit flawed credential management.

Common issues:

- Weak password resets
- Missing security controls
- Not using OAuth, no 2FA

Fix:
Strong authentication flows, MFA, token expiration/rotation, strict JWT validation and never assume that APIs are hidden.

API3 Broken Object Property Level Authorization

This is also known as “Excessive Data Exposure.
APIs return more data than necessary, exposing sensitive fields even if the UI hides them.

Fix:
Use server-side response filtering, not client-side filtering and return only required data.

API4 Unrestricted Resource Consumption

Attackers overwhelm API servers with expensive operations or large volumes of requests.

Examples:

- No resource rate limiting
- Unlimited file uploads
- Costly database queries

Fix:
Rate limiting, input validation, request size limits and traffic control.

API5 Broken Function Level Authorization (BFLA)

Attackers call privileged API endpoints (example admin functions) without proper authorization checks.

Fix:
Role-Based Access Control (RBAC) and always validate permissions server-side as well as identifying functions with sensitive capabilities.

API6 Unrestricted Access to Sensitive Business Flows

Attackers exploit API workflows intended for humans and automate them and also abuse them.

Fix:
Add behavioral checks, anti-automation controls, avoid incremental ids and introduce tile out for codes.

API7 Server-Side Request Forgery

APIs are tricked to go somewhere else. An attacker might exploit input and make a request to go to a malicious 3rd party..

Common issues:

- Not validating inputs

Fix:
Validate inputs, apply least privilege, simulate SSRF in your API during Quality Assurance test, validate URLs and block internal IP ranges.

API8 Security Misconfiguration

The use of default settings, unnecessary features and lack of proper policies which exposes risks.

Risk areas:

- Lack of security hardening
- When error messages are too revealing
- Blind trust in upstream API responses

Fix:
Helpful and not revealing error messages, enforcing COR and rate limiting.

API9 Improper Inventory Management

Organizations lack visibility into all API versions, endpoints and environments.

Risks:

- Forgotten older API versions still online
- Shadow APIs not monitored
- Inconsistent security controls across versions
- Outdated documentation

Fix:
 Create an API inventory, enforce versioning standards, deprecate unused APIs, identify internal and 3rd party APIs.

API10 Unsafe Consumption of APIs

This is almost the opposite of API9 but it deals with 3rd party APIs which may have vulnerabilities.

Fix:
Do not assume that 3rd party APIs are safe, validate returning data, check 3rd party pentesting reports

4. The three pillars of API Security

Governance (policies and processes), Testing (continuous security testing), and Monitoring (detecting abnormal activity) are the 3 pillars of API Security

5. API Security technology landscape

Here I got an overview of tools such as API gateways, Web Application Firewalls, authentication systems and monitoring tools as well as how they fit into a layered security model.

6. Best Practices & do's and don'ts

In this section I got an understanding of secure design, input validation, rate limiting, strict authorization, secure logging and general team-based security strategies for API security.

- Do not trust anything (inputs or networks)
- Do validate all inputs
- Do not reveal useful information in error messages
- Do expect attackers to find APIs
- Do not have hidden or unadvertised features
- Do not filter data on the UI, control it at application level
- Do use API Gateways to control access
- Do not forget Authorization and Authentication
- Do require API documentation
- Do continuous testing, pre-production

7. Strategic insight

Security must be built into the API development lifecycle rather than added later. The course emphasized building a full API security program.

Summary

APIs represent a major attack surface and require a layered security approach. Understanding real-world breaches, the OWASP API Top 10 and the pillars of governance, testing and monitoring provides a solid foundation for designing, assessing and securing APIs effectively.

Blockers

I did not know whether I should include what I studied in module one at API Sec University in this report. The slack channel announcement says only lab the report(the Swagger file) but during Saturday's live lesson the answer was different and I did not get clarification. I ended up just including it and my badge as shown in the report templates that were provided.

End of report